Lab Experiment 10: Understanding Node.js Modules and HTTP Server with AI Assistance

Learning Outcome

At the completion of this lab experiment, students should be able to:

- Understand the fundamentals of Node.js modules.
- Import and export custom modules in Node.js.
- Use built-in Node.js modules.
- Create a basic HTTP server using Node.js.
- Handle server requests and responses.
- Run and test a local Node.js server.
- Use AI tools to assist with coding, debugging, and understanding server-side JavaScript concepts.

Introduction

Node.js allows developers to use JavaScript outside the browser to build powerful back-end applications and servers.

One of the core concepts in Node.js is the use of modules. Modules help organize code into reusable components, making applications easier to maintain and scale.

In this lab, students will explore:

- Creating custom modules
- Using built-in modules
- Exporting and importing functions
- Building a simple HTTP server
- Returning responses to users through the browser

Students will also use AI tools to assist with development and debugging.

AI Assistance Activity

Use AI tools such as:

- OpenAI ChatGPT
- GitHub Copilot
- Gemini

Use AI assistance to help with:

- Understanding Node.js syntax
- Generating starter code
- Explaining modules and exports
- Debugging server errors
- Improving server responses
- Understanding HTTP requests and responses

Example AI Prompts

- "Explain how Node.js modules work."
- "Generate a simple Node.js HTTP server example."
- "Show how to export and import functions in Node.js."
- "Help me debug a Node.js server error."
- "Create a custom math module for Node.js."

Objective

Build a simple Node.js application that:

- Uses custom and built-in modules
- Creates a local HTTP server

- Responds to browser requests
- Displays custom content in the browser

Assignment Requirements

Your project must include:

1. **Module Requirements**
    - Create at least one custom module
    - Export at least two functions from the module
    - Import and use the module in the main application file
    - Use at least one built-in Node.js module

2. **HTTP Server Requirements**

    Create an HTTP server that:

    - Runs on a local port
    - Responds to browser requests
    - Displays HTML content in the browser
    - Includes at least three routes/pages
    - Displays different content for each route

3. **Required Routes**

    Your server should include:

    - Route: /, description: Home page
    - Route: /about, description: About page
    - Route: /contact, description: Contact page
    - Route: 404, description: Invalid route page

Each Page Must Include

- A page title
- A heading
- A paragraph with interesting or curious information related to Node.js or web servers

4. **Styling Requirements**
    - Include basic HTML formatting
    - Use inline CSS or external CSS
    - Make pages visually organized and readable

Lab Description

In this lab, students will build a basic Node.js web server application while learning how modules work.

The application will include:

- A custom utility module
- A simple HTTP server
- Multiple routes
- Dynamic responses
- AI-assisted debugging and code generation

Students will gain practical experience with back-end JavaScript concepts used in real-world web development.

Recommended Software

Install the following software before starting:

- Node.js Official Website
- Visual Studio Code

Verify installation:

```
node -v
```

Check npm version:

```
npm -v
```

Suggested Project Structure

```
node-lab/
│
├── app.js
├── mathModule.js
├── utils.js
├── package.json
└── style.css
```

Lab Procedure

Part 1 – Initialize the Project

Create a new project folder:

```
mkdir node-lab
```

Navigate into the folder:

```
cd node-lab
```

Initialize the project:

```
npm init -y
```

Part 2 – Create a Custom Module

Create a file named:

```
mathModule.js
```

Example functions:

```
function add(a, b) {
  return a + b;
}

function multiply(a, b) {
  return a * b;
}

module.exports = {
  add,
  multiply
};
```

Part 3 – Import the Module

Inside app.js:

```
const math = require('./mathModule');

console.log(math.add(5, 3));
console.log(math.multiply(4, 2));
```

Run the file:

```
node app.js
```

Part 4 – Create an HTTP Server

Use the built-in HTTP module:

```
const http = require('http');
```

Create a server:

```
const server = http.createServer((req, res) => {
  res.write('Hello from Node.js Server');
  res.end();
});
```

Set a port and listen:

```
server.listen(3000, () => {
  console.log('Server running on port 3000');
});
```

Part 5 – Add Multiple Routes

Use conditional statements to create routes:

```
if (req.url === '/') {
  res.write('<h1>Home Page</h1>');
}
else if (req.url === '/about') {
  res.write('<h1>About Page</h1>');
}
else {
  res.write('<h1>404 Page Not Found</h1>');
}
```

Part 6 – Add Curious Notes

Each page should contain an interesting fact.

Example Ideas

- Node.js uses an event-driven architecture.
- JavaScript can run outside the browser using Node.js.
- HTTP servers respond to requests from clients such as browsers.
- Node.js is commonly used for APIs and real-time applications.
- Modules help organize reusable code.

Part 7 – Add Styling

Add simple HTML and CSS formatting.

Example:

```
res.write(`
  <html>
    <head>
      <title>Home</title>
    </head>
    <body>
      <h1>Welcome to Node.js</h1>
      <p>Node.js allows JavaScript to run on servers.</p>
    </body>
  </html>
`);
```

Testing the Server

Start the server:

```
node app.js
```

Open the browser:

```
http://localhost:3000
```

Test all routes:

- /
- /about
- /contact
- Invalid route example: /test

Deliverables

Students must submit:

1. Complete Node.js project files
2. Screenshots of:
    - Home page
    - About page
    - Contact page
    - 404 page
3. GitHub repository link
4. Create a README.md file and write reflection document describing:
    - What was learned
    - Challenges encountered
    - How AI assistance was used

Reflection Questions

Answer the following questions in the README.md file:

1. What is the purpose of modules in Node.js?
2. Why are HTTP servers important in web development?
3. What is the difference between client-side and server-side JavaScript?
4. How did AI tools help during development?
5. What improvements would you make to your server?

Conclusion

This lab introduces foundational back-end development concepts using:

- Node.js

- HTTP servers
- Modules
- Routing
- AI-assisted development workflows

By completing this exercise, students will gain practical experience building server-side JavaScript applications and preparing for more advanced full-stack development topics.

Grading Rubric

- Custom module created correctly: 15%
- Functions exported/imported properly: 15%
- HTTP server works correctly: 20%
- Multiple routes implemented: 15%
- 404 page implemented 10%
- Styling and page formatting 10%
- Curious notes included 8%
- AI assistance reflection question 7%